Swish : Private Social Sharing Platform for Campus Communities

Software Requirements Specification

Document ID	SOC-WEB-2026-094-SRS-v1.2
Project Code	SOC-WEB-2026-094
Classification	Internal - Restricted
Version	1.2
Prepared By	Product Engineering
Reviewed By	Technical Lead, Delivery

Revision History

Version	Summary of Change	Author
1.0	Initial draft circulated for internal review	Product Engineering
1.1	Scope refined following technical review	Technical Lead
1.2	Functional requirement identifiers, non-functional requirements and acceptance criteria added	Product Engineering

---

1. Overview

Swish is a full-stack web application that brings the character of social media into a secure, campus-exclusive environment. The platform allows students and faculty to share photos, updates and achievements within their academic network, while ensuring privacy and moderation suitable for institutional use.

The objective is a digital space that celebrates campus life, where posts, comments and likes connect people within the same community.

2. Project Goals

- Build a MERN-based social networking platform limited to verified campus users.
- Enable users to post images, captions, and short updates visible to their network.
- Implement engagement features including likes, comments, and a follow system.
- Provide an admin dashboard for content monitoring and moderation.

- Deliver a modern, mobile-responsive interface with a clean, minimal aesthetic.

3. Technical Scope

The solution shall be developed using **MongoDB, Express, React, and Node.js**. **JWT**-based authentication and role-based access control shall be implemented for Students, Faculty and Administrators. Secure file handling for image uploads shall be implemented using **Multer** or **Cloudinary**.

4. Functional Requirements

Priorities follow the MoSCoW convention. **Must** requirements are mandatory for acceptance; **Should** requirements are expected unless justified; **Could** requirements are desirable enhancements.

ID	Requirement	Priority
FR-01	The system shall provide registration and login restricted to verified campus users.	Must
FR-02	The system shall allow each user to maintain a profile with a profile photo, biography and basic information.	Must
FR-03	The system shall allow users to create image-based posts with captions.	Must
FR-04	The system shall present a feed of posts to the user.	Must
FR-05	The system shall allow users to like a post.	Must
FR-06	The system shall allow users to comment on a post.	Must
FR-07	The system shall allow users to follow other users.	Must
FR-08	The system shall provide an explore section presenting trending posts or profiles.	Should
FR-09	The system shall provide administrators with controls to moderate content, handle reports and manage user activity.	Should
FR-10	The system shall issue alerts for likes, comments and follows.	Could
FR-11	The system shall handle image uploads securely.	Must
FR-12	The system shall present a mobile-responsive interface.	Must

5. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	The application shall respond to user actions without a delay noticeable during normal use.
NFR-02	Scalability	The application shall remain responsive as the number of records and concurrent users grows.
NFR-03	Security	All API endpoints shall enforce authentication and role-based authorisation. Uploaded files shall be validated before storage, and access shall be restricted to verified campus users. Credentials and connection strings shall not be committed to source control.
NFR-04	Reliability	Data submitted by users shall be persisted such that work is not lost in the event of an unexpected disconnection.
NFR-05	Usability	The interface shall be operable without prior training and shall provide visible feedback for every user action.
NFR-06	Compatibility	The application shall function on current versions of major desktop browsers and shall remain usable at tablet screen sizes.
NFR-07	Accessibility	Interactive controls shall be keyboard reachable and clearly labelled.
NFR-08	Maintainability	Code shall follow a consistent style with modular components, and environment setup shall be documented in the repository.

6. Acceptance Criteria

The following criteria shall be demonstrated during user acceptance testing.

ID	Ref	Criterion
AC-01	FR-01	**Given** a user who is not a verified campus user, **when** they attempt to register, **then** access is refused.
AC-02	FR-03	**Given** a logged-in user, **when** they publish a post with an image and caption, **then** the post is stored and appears in the feed.
AC-03	FR-05	**Given** a post in the feed, **when** a user likes it, **then** the like is recorded and the updated count is displayed.
AC-04	FR-06	**Given** a post in the feed, **when** a user submits a comment, **then** the comment is stored and displayed against that post.

AC-05	FR-07	**Given** two registered users, **when** one follows the other, **then** the relationship is recorded and reflected on both profiles.
AC-06	FR-09	**Given** a reported post, **when** an administrator removes it, **then** the post no longer appears in the feed.

7. Mid-Point Review

A mid-point review shall be conducted at approximately 60% of the delivery period. Each team shall submit the following:

- **Repository link** — public, or with access granted to the review team, containing the complete commit history for the project to date.
- **Demo video** — a screen recording of three to five minutes showing the running application. Slides and design mockups shall not be accepted in place of a recording.
- **Status note** — a short summary of completed requirements, work in progress, and any blockers or scope concerns.

The submission shall be assessed against the Must-priority requirements listed in Section 4. The following shall be demonstrable at the mid-point review: **FR-01, FR-03, FR-04, FR-11**.

Commit history shall be reviewed for evidence of contribution distributed across the team and of sustained work across the delivery period.

Requirements not evidenced at the review shall be recorded and re-planned across the remaining delivery period. Review outcomes shall be communicated to the team leader.

8. Expected Deliverables

- A functional prototype of Swish demonstrating posting, feed, and interaction features.
- RESTful backend APIs for authentication, posts, and comments.
- A React frontend with responsive design and fluid navigation.
- Documentation covering data models, API endpoints, and architecture.
- A demonstration highlighting the user experience and technology stack.

9. Expected Outcomes

- A campus-exclusive social platform encouraging creative expression and engagement.
- Consolidated understanding of authentication, file uploads, and frontend-backend data flow.
- Improved capability in scalable web architecture and interface design.
- Practical experience in building social-network-style applications.
- A deployable, visually appealing product suitable for institutional environments.

10. Approach Guidelines

- Define collections for Users, Posts, and Comments before backend setup.
- Implement authentication and post creation as core modules before adding engagement features.
- Prioritise responsive interface design and intuitive navigation.
- Conduct basic testing for performance, media handling, and security.
- Prepare a final demonstration showing both user and administrator perspectives.